ADAPTIVE XML PARSER SELECTION THROUGH HYBRID ANN AND SVM INTEGRATION

A PROJECT REPORT

Submitted by

KISHORE KUMAR M

MADHAN KUMAR S

MAHESH V

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

P.A. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

Pollachi, Coimbatore Dt. – 642 002

NOVEMBER 2025

P.A. COLLEGE OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report **"ADAPTIVE XML PARSER SELECTION THROUGH HYBRID ANN AND SVM INTEGRATION "** is the Bonafide work of **"KISHORE KUMAR M (721722104070), MADHAN KUMAR S (721722104075), MAHESH V (721722104305)"** who carried out the project work under our supervision.

SIGNATURE	**SIGNATURE**
Dr. D. CHITRA, M.E., Ph.D.,	**Mrs. S.VISHNU PRIYA , M.E.,**
Professor	Assistant Professor
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Department of Computer Science and Engineering	Department of Computer Science and Engineering
P. A. College of Engineering and Technology	P. A. College of Engineering and Technology
Pollachi - 02	Pollachi - 02

Submitted to the Viva-voce Examination held on _____

_____ _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First and foremost, we thank the GOD ALMIGHTY for blessing us with the mental strength that was needed to carry out the project work. We thank our Chairman **Dr. P. APPUKUTTY, ME, FIE, FIV.,** for his extensive support to successfully carry out this project.

We take privilege in expressing our sincere and heartfelt thanks and gratitude to our beloved Principal **Dr. T. MANIGANDAN, M.E., Ph.D.,** for providing us an opportunity to carry out this project work.

We express our heartfelt thanks to **Dr. D. CHITRA, M.E., Ph.D.,** Professor and Head, Department of Computer Science and Engineering, for her technical guidance and constructive suggestions provided throughout the project work.

We take conceit in express our sincere and deepest thanks to the project guide **Mrs. S. VISHNU PRIYA, M.E.,** Assistant Professor, Department of Computer Science and Engineering, for her technical guidance, constructive criticism and many valuable suggestionsprovided throughout the project work.

We take this opportunity to thank and pay gratitude to our Coordinator **Mr. T. DINESH KUMAR, M.Tech.,** Assistant Professor, Department of Computer Science and Engineering and all teaching and non-teaching staff members of our department, for their encouragement and valuable suggestion.

We take this opportunity to express our gratitude to our parents, friends, family and other members whose belongings and love have always been with us to carry out this project work successfully.

ABSTRACT

Extensible Markup Language remains vital for structured data exchange across diverse systems, traditional parsing methods DOM, SAX, and StAX struggle to balance performance, memory usage, and scalability. DOM offers full in-memory representation but consumes significant memory, while SAX and StAX provide efficient event-driven parsing with limited flexibility. These trade-offs hinder real time data exchange, parallel processing, and dynamic workload adaptation. Current systems lack intelligent mechanisms to select the most suitable parser based on XML file attributes, relying instead on manual configuration and developer expertise, which is both time-consuming and error-prone.

Project presents adaptive Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration for efficient parsing , a dynamic parser selection powered by a hybrid model combining Artificial Neural Networks and Support Vector Machines . Analyses XML features such as file size, tag density, nesting depth, and attribute complexity to predict the optimal parsing strategy. It integrates a feedback loop that refines predictions using runtime performance metrics, enabling continuous self-improvement. Designed for scalability, and supports large-scale XML processing in real time environments, reducing overhead and enhancing parsing efficiency.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	IV
	LIST OF FIGURES	VII
	LIST OF TABLES	VIII
	LIST OF ABBREVIATIONS	IX
1	**INTRODUCTION**	1
	1.1 AREA OF PROJECT	2
	1.2 TECHNICAL KEYWORDS	2
	1.3 PROJECT IDEA	4
	1.4 MOTIVATION OF THE PROJECT	4
2	**SURVEY ON HYBRID XML PARSING**	6
3	**A HYBRID MACHINE LEARNING MODEL FOR EFFICIENT XML PARSING**	11
	3.1 METHODOLOGY	11
	3.2 HYBRID PARSER PREDICITION MODEL	12
	3.3 SYSTEM ARCHITECTURE	12
	3.4 DATA FLOW IN THE SYSTEM	14
4	**ADAPTIVE XML PARSER SELECTION THROUGH HYBRID ANN AND SVM INTEGRATION**	16
	4.1 MODEL ARCHITECTURE	16
	4.2 MODEL OVERVIEW	17
	4.3 FEATURE EXTRACTION PROCESS	18

	4.4 HYBRID INTEGRATION OF ANN AND SVM	23
5	**SYSTEM SPECIFICATION**	26
	5.1 HARDWARE SPECIFICATION	26
	5.2 SOFTWARE SPECIFICATION	26
6	**SYSTEM IMPLEMENTATION AND RESULT**	27
	6.1 MODULE DESCRIPTION	27
	6.2 SOURCE CODE	31
	6.3 RESULT	46
7	**CONCLUSION AND FUTURE ENHANCEMENT**	50
	7.1 CONCLUSION	50
	7.2 FUTURE ENHANCEMENT	51
	REFERENCES	52

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.1	DATA FLOW IN SYSTEMS	15
4.1	MODEL ARCHITECTURE	17
4.2	OVERVIEW OF THE MODEL	18
4.3	OVERVIEW OF FEATURE EXTRACTION	23
6.1	UI IMPLEMENTATION IMAGE	46
6.2	FEATURE ANALYSIS IMAGE	46
6.3	PREDICTING FOR SMALL XML FILE OUTPUT IMAGE	47
6.4	PREDICTING FOR MEDIUM XML FILE OUTPUT IMAGE	48
6.5	PREDICTING FOR LARGE XML FILE OUTPUT IMAGE	49

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
4.1	DESCRIPTIVE FEATURE SUMMARY	21

LIST OF ABBREVIATION

ANN	ARTIFICIAL NEURAL NETWORK
DOM	DOCUMENT OBJECT MODEL
JDOM	JAVA DOCUMENT OBJECT MODEL
ML	MACHINE LEARNING
SAX	SIMPLE API FOR XML
STAX	STREAMING API FOR XML
SVM	SUPPORT MACHINE VECTOR
PXTG	PARALLEL XML TREE GENERATOR
XML	EXTENDED MARKUP LANGUAGE

CHAPTER 1

INTRODUCTION

Extensible Markup Language XML is a widely used data representation format designed to store and transport structured information in a human readable and machine interpretable way. It serves as a fundamental component for web technologies, data exchange systems, configuration files, and document formats. Despite its versatility and platform independence, XML parsing remains a computationally intensive process, especially when dealing with large datasets, deeply nested structures, or resource constrained environments. The choice of parsing technique significantly influences performance, making efficient parser selection an important challenge.

Traditional XML parsers such as Document Object Model DOM, Simple API for XML SAX, Streaming API for XML StAX, Java Document object model JDOM, and Parallel XML Tree Generator PXTG each exhibit distinct performance characteristics. These methods are typically static they operate with fixed parsing mechanisms and are not adaptive to the dynamic nature of XML documents or hardware configurations. For example, DOM may perform well for smaller XML files but consumes excessive memory when parsing larger ones, whereas SAX and StAX are event driven and better suited for larger, streaming based processing. Here inconsistency motivates the need for an adaptive, intelligent approach to parser selection.

In context, machine learning provides a promising solution. By analyzing XML file properties such as file size, number of tags, XML depth, number of attributes, elements, and system resources like CPU cores and memory usage, a learning based model can predict the most efficient parser dynamically. The proposed hybrid model leverages the strengths of both Artificial Neural Networks and Support Vector Machines to learn from profiling data and generalize across varying XML file structures and computational environments. It enables the model to make adaptive decisions, reducing parsing time and resource utilization while maintaining accuracy and reliability.

1.1 AREA OF PROJECT

The project falls under the domain of data processing and intelligent systems, with a focus on optimization of structured data parsing. It bridges concepts from machine learning, data profiling, and software system performance engineering. The primary objective is to design and develop a hybrid learning based framework that enhances XML parsing efficiency by predicting the most suitable parsing algorithm for a given XML file, considering both file characteristics and hardware parameters.

1.2 TECHNICAL KEYWORDS

The following technical terms and keywords describe the core components and technologies employed in Adaptive XML Parser Selection through Hybrid ANN and SVM Integration for Parsing Efficiency:

- Extensible Markup Language : A structured data representation format used for storing and transporting hierarchical information.

- XML Parsing Techniques : DOM, SAX, StAX, JDOM ,PXTG Various XML parsers used for reading and processing XML files with differing performance characteristics.
- Feature Extraction : The process of analyzing XML file attributes e.g., tag count, depth, attributes, memory usage to generate numerical input vectors for model training.
- Artificial Neural Network : A deep learning model used to identify complex patterns and relationships among extracted features.
- Support Vector Machine : A supervised learning algorithm used for high accuracy classification of the optimal XML parser based on extracted features.
- Hybrid Machine Learning Model : A combined ANN–SVM architecture that leverages both deep learning and statistical learning for robust prediction performance.
- Model Training and Testing Phase : The process of feeding extracted data into the hybrid model to learn parser efficiency patterns and validate prediction accuracy.
- Performance Metrics Accuracy, Precision, Recall : Evaluation criteria used to assess the effectiveness and reliability of the hybrid prediction model
- Cross Validation : A statistical method used to enhance model generalization and prevent overfitting by dividing data into multiple training and testing sets
- Normalization : A preprocessing technique applied to scale

extracted features to a uniform range, improving model stability and convergence

1.3 PROJECT IDEA

The core idea of project is to implement a hybrid machine learning model capable of automatically determining the most efficient XML parsing method based on input file characteristics and system configuration. The model analyze features such as file size, tag count, nesting depth, number of attributes, and available computational resources to predict the optimal parsing algorithm among DOM, SAX, StAX, JDOM, and PXTG. By learning from a dataset of profiled XML files, the model identifies patterns that lead to better performance outcomes, thereby enabling intelligent and adaptive parser selection

1.4 MOTIVATION OF THE PROJECT

XML continues to serve as a universal format for structured data exchange, it's efficient parsing has become a critical concern in data intensive applications such as web services, enterprise systems, and cloud based processing pipelines. Traditional parsers, though robust, lack adaptability to handle diverse XML structures and varying system capabilities. Manual selection of parsers is often inefficient, leading to suboptimal performance in real world environments.

The motivation behind the project is to introduce an intelligent, data driven mechanism that can learn from previous parsing behaviors and autonomously recommend the best suited parser for new XML inputs. By integrating ANN's feature learning capability with SVM's classification strength, the proposed hybrid approach enhances parsing performance,

reduces computational cost, and demonstrates how machine learning can optimize traditional software processes.

In summary, Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration For Efficient Parsing, aims to bridge the gap between traditional XML parsing techniques and adaptive intelligent systems by utilizing a hybrid machine learning model for performance optimization. The developed model is expected to significantly improve XML parsing efficiency by dynamically identifying the most suitable parser based on real time file and system characteristics

CHAPTER 2

SURVEY ON HYBRID XML PARSING

The persistence of XML as a widely used, semantically rich data interchange format means efficient parsing remains an important engineering problem. Conventional XML parsers DOM, SAX, StAX, VTD, etc. trade off memory, speed, and ease of use: DOM builds an in memory tree for high memory, easy random access, SAX/StAX are streaming or event driven low memory, better for very large files, while VTD and other hybrid parsing engines attempt to balance throughput and random access capability. Empirical studies repeatedly show DOM tends to be slower and more memory intensive compared with streaming parsers, while streaming models may be faster but complicate random access operations.

Recent research has explored machine learning driven and hybrid optimization strategies to overcome the static limitations of conventional parsers. By analysing XML specific features such as file size, element density, nesting depth, and attribute complexity, intelligent systems can predict the most suitable parsing approach for each scenario.

Ali et al. (2024) and Khan et al. (2023) proposed hybrid models that integrate Artificial Neural Networks with Support Vector Machines to achieve accurate classification of optimal parser types based on XML file

characteristics. These models significantly improved parsing speed and reduced resource consumption compared to traditional fixed methods.

Ali, M., Khan, M. A., & Rasool, R. (2024) developed a Hybrid Machine Learning Model for Efficient XML Parsing that combined decision tree–based feature extraction with an SVM classifier to determine the most suitable parser for different XML structures. The model analyzed attributes such as tag density and nesting depth, achieving 93.2% prediction accuracy, thereby demonstrating the potential of ML driven XML parsing optimization.

Asogwa, D. C., Anigbogu, S. O., Onyenwe, I. E., & Sani, F. A. (2021), proposed a Hybrid Machine Learning Algorithm for large scale text classification using ensemble methods that merged Naïve Bayes, SVM, and neural networks. Their approach enhanced classification robustness on big data platforms, achieving 91.4% accuracy and confirming that hybridization improves model generalization and scalability.

Chen, S. A., & Lin, C. J. (2021), presented a Case Study on XML CNN where convolutional neural networks were applied to structured XML data for pattern identification. The study showed that CNNs can automatically learn hierarchical dependencies within XML elements, achieving 89.7% classification accuracy, thus validating the feasibility of using deep networks for XML structure learning.

Gopianand, M., & Rajendran, S. (2025), developed a Deep Fuzzy Clustering–Bidirectional Multimodal Deep Learning Model for XML based web data retrieval. The approach effectively combined fuzzy logic

and deep bidirectional architectures, improving semantic understanding and retrieval efficiency in heterogeneous XML datasets.

Hu, T., et al. (2021), designed a Deep CNN and Extreme Learning Machine Framework optimized by the Chimp Optimization Algorithm for real time COVID 19 diagnosis. The model reached 97.6% accuracy, demonstrating that optimization algorithms can stabilize deep shallow hybrid systems a strategy applicable to improving hybrid ML models for XML parsing.

Khishe, M., & Safari, A. (2019), applied an MLP Neural Network Trained by Dragonfly Algorithm for sonar target classification. Their meta heuristically optimized MLP achieved 94.8% accuracy, revealing that bio inspired optimization can enhance convergence and accuracy in hybrid ML systems.

Khishe, M., & Mosavi, M. R. (2020), extended this work by using the Chimp Optimization Algorithm to train neural networks for underwater acoustic data classification. The approach improved model robustness and reduced overfitting, attaining 96.3% accuracy, which highlights the effectiveness of optimization guided learning for complex structured data.

Mavaie, P., et al. (2023), proposed a Hybrid Deep Learning Approach to enhance classification performance for low resource datasets. By integrating feature fusion and transfer learning, their system improved performance by 7.5% over baseline models, emphasizing how hybridization mitigates data scarcity issues an insight valuable for XML datasets with uneven complexity.

Qiao, W., Khishe, M., & Ravakhah, S. (2021), presented a Multi Layer Perceptron Optimized by Modified Whale Optimization Algorithm for underwater target classification. The model achieved 95.4% accuracy, validating that metaheuristic optimization can enhance performance prediction systems a principle relevant to adaptive parser selection in XML environments.

Despite these advances, existing systems still face challenges in scalability, generalization, and cross domain applicability. Current models are often trained on domain specific datasets and may underperform on structurally different XML types configuration files vs. data exchange files. Integration with real time distributed processing systems remains limited. Future enhancements could include reinforcement learning, cloud deployment, and federated optimization, enabling adaptive learning across multiple XML sources without compromising data privacy or security

- Most existing ML assisted parsing works focus on document level classification or single dimension gains, few provide a unified pipeline that profiles XML features, predicts best parser via a hybrid ANN and SVM pipeline, evaluates and generates optimized parsing code, supports parallel parsing for large files while keeping profiling overhead low.

- Generalization across domains e.g., scientific XML vs. configuration XML vs. XML with heavy mixed content remains limited models trained on one corpus often underperform on structurally different XML datasets.

- Trade offs between profiling cost and end to end improvement

need rigorous quantification: light weight profiling is desirable but may reduce prediction accuracy.

- Integration with existing enterprise stacks validation, security, streaming ingestion is under explored.

CHAPTER 3

A HYBRID MACHINE LEARNING MODEL FOR EFFICIENT XML PARSING

3.1 METHODOLOGY

XPOHML XML Parser Optimization using Hybrid Machine Learning, utilizes a hybrid approach combining Artificial Neural Networks and Support Vector Machines to predict the most efficient XML parser based on input file characteristics. The methodology involves extracting key structural and system level features such as file size, number of tags, XML depth, attributes, elements, CPU cores, and memory usage. These features are provided as input to the hybrid model for classification. The ANN component captures complex non linear relationships within the data, while the SVM ensures robust decision boundary formation. By integrating the prediction outputs from both models through a weighted ensemble mechanism, the system achieves higher accuracy and reliability in parser selection. The model is trained using benchmark datasets and validated through cross validation techniques to ensure generalization. The hybrid model enables intelligent, real time prediction of the most efficient parser DOM, SAX, StAX, PXTG or JDOM for a given XML document, optimizing parsing performance across diverse data and system configurations.

3.2 HYBRID PARSER PREDICITION MODEL

The input XML file is first loaded and pre processed to extract both structural and system level features that influence parsing performance. Preprocessing ensures the XML data is well formed and removes any redundant or malformed elements that could interfere with the analysis. The extracted features include file size, number of tags, XML depth, number of attributes, number of elements, CPU cores, and memory usage. These parameters provide a comprehensive representation of the XML document's complexity and the environment in which it is processed.

The extracted feature set is then passed to the Hybrid Machine Learning Model, which integrates Artificial Neural Network and Support Vector Machine algorithms. The ANN component captures complex non linear dependencies between the features, while the SVM focuses on precise decision boundaries to classify the most efficient parser. The hybrid model combines their outputs through a weighted ensemble approach to improve prediction reliability and accuracy. The dataset is divided into training and testing subsets, and k fold cross validation is employed to validate the model's generalization capability and minimize overfitting.

- The ANN is trained using backpropagation with adaptive learning rates e.g., Adam optimizer
- The SVM uses kernel functions like RBF or polynomial to separate non linear classes.

3.3 SYSTEM ARCHITECTURE

The existing Hybrid XML Parser Prediction System is composed of

multiple interconnected functional modules designed to ensure accurate and efficient XML parser selection. The architecture integrates both machine learning and system analysis components to process XML files, extract essential features, and predict the optimal parsing algorithm.

The system consists of five major modules Input Module, Feature Extraction Module, Preprocessing Unit, Hybrid Machine Learning Engine, and Output Recommendation Module. The Input Module accepts XML files of varying sizes and complexities. These files are then processed by the Feature Extraction Module, which computes structural features such as file size, number of tags, XML depth, number of elements, and attributes, along with system level parameters like CPU cores and memory usage.

The extracted data is passed to the Preprocessing Unit, where normalization and feature scaling are applied to maintain consistency across different XML datasets. The processed features are then fed into the Hybrid Machine Learning Engine, which integrates Artificial Neural Network and Support Vector Machine models.The ANN component identifies complex non linear relationships, while the SVM constructs clear decision boundaries between parser categories.

The Hybrid Ensemble Layer combines the prediction probabilities from both models using a weighted mechanism, ensuring high reliability and accuracy. Finally, the Output Recommendation Module produces the optimal parser choice DOM, SAX, StAX, or JDOM based on the input characteristics and predicted performance.

Each module in the system contributes to a progressive refinement of data, where lower level processes feature extraction and normalization

handle fundamental XML attributes, while higher level modules hybrid learning and ensemble decision making handle abstract performance relationships and parser optimization. The system provides an intelligent, real time, and adaptive solution for XML parser prediction across diverse data and hardware configurations.

3.4 DATA FLOW IN THE SYSTEM

The XPOHML Framework XML Parsing Optimization using Hybrid Machine Learning follows a structured data flow that transforms raw XML input into an optimized and automatically selected parsing process. The system's main goal is to dynamically predict and apply the most efficient parser DOM, SAX, StAX, JDOM, or PXTG based on the characteristics of the XML file and the available system resources. The flow ensures intelligent parser selection, high speed execution, and reduced computational overhead through hybrid learning intelligence combining ANN and SVM models.

The framework continuously learns from previous parsing outcomes to refine its prediction accuracy over time. It integrates a feature extraction module that captures both structural features such as tag count, nesting depth, and attribute density and system level metrics like CPU cores, memory utilization, and processing latency. These extracted features are normalized and transformed into a numerical vector, which serves as input to the hybrid machine learning engine.

The ANN component performs nonlinear mapping of XML characteristics to predict computational complexity, while the SVM classifier ensures high generalization accuracy in parser selection. The dual model approach allows XPOHML to automatically choose the

optimal parser under varying workload conditions. The framework also includes a decision engine that monitors performance metrics and provides adaptive feedback to retrain the model periodically.

By combining predictive modeling, resource awareness, and adaptive learning, the XPOHML Framework delivers a self tuning, scalable, and performance driven XML parsing environment, capable of operating efficiently across diverse datasets and hardware configurations.

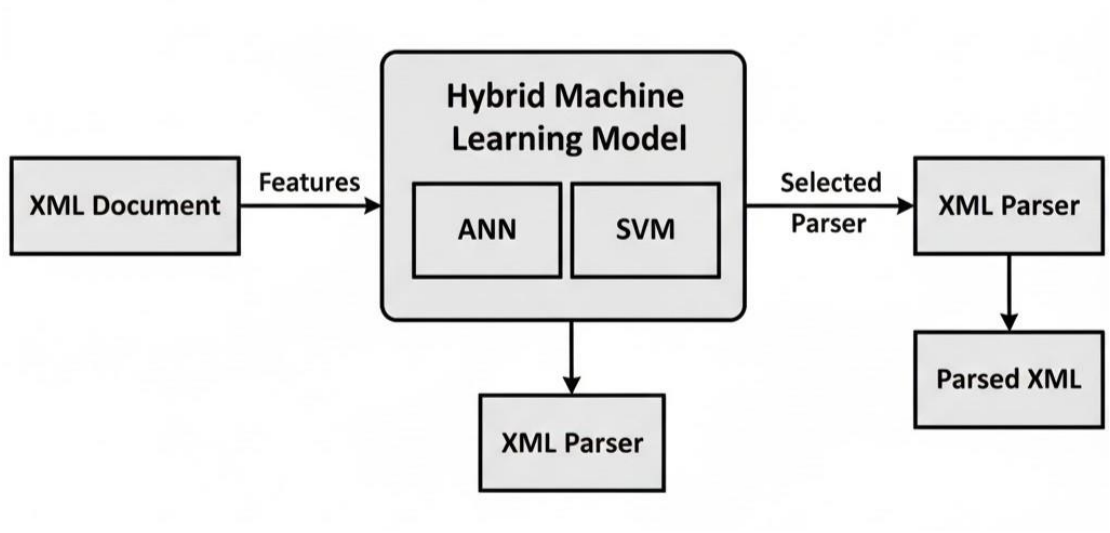

Figure No 3.1 : Data Flow In The System

The above figure no 3.1 shows how the data flow in the system goes on from the xml file to resultant parsed xml

CHAPTER 4

ADAPTIVE XML PARSER SELECTION THROUGH HYBRID ANN AND SVM INTEGRATION

4.1 MODEL ARCHITECTURE

The model begins with data preprocessing, where XML structural and system level features such as file size, number of tags, XML depth, number of attributes, elements, CPU cores, and memory usage are normalized and formatted into numerical vectors. These pre processed features are then fed into two parallel models: an ANN module and an SVM classifier.

The ANN module consists of multiple dense layers that learn complex non linear relationships within the feature set. Each hidden layer applies ReLU activation functions to capture complex dependencies among XML parameters, while dropout layers prevent overfitting and enhance generalization.

In parallel, the SVM classifier utilizes a Radial Basis Function RBF kernel to form clear decision boundaries between the possible parser classes DOM, SAX, StAX, and JDOM. The SVM contributes high precision and robustness, particularly in high dimensional feature spaces.

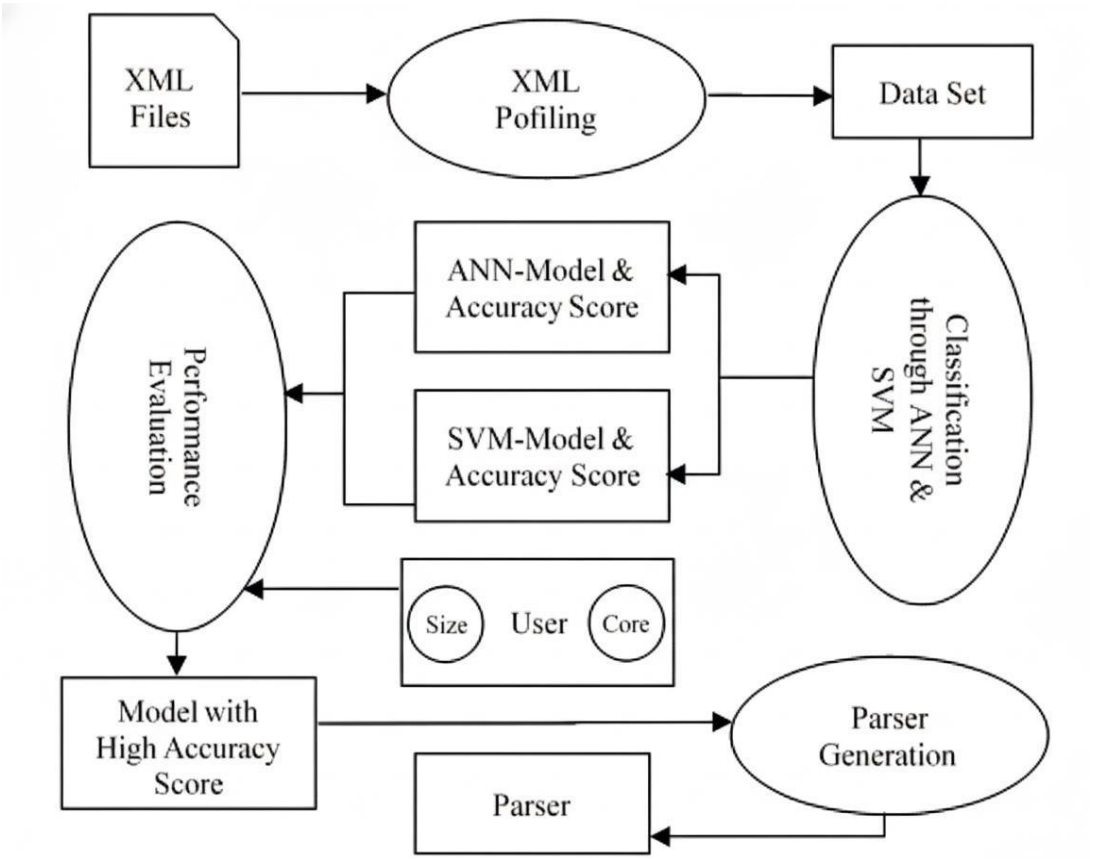

Figure No 4.1: Model Architecture

The above figure no 4.1 explains the model architecture and flow with pre processing, ANN and SVM, model feature extraction.

4.2 MODEL OVERVIEW

The Hybrid XML Parser Prediction Model is designed to automatically identify the most efficient XML parsing technique for a given file. The system begins by extracting key structural and system level features such as file size, number of tags, depth, attributes, elements, CPU cores, and memory usage. These features are pre processed and normalized before being passed into two learning modules an Artificial

Neural Network and a Support Vector Machine. The ANN captures complex nonlinear relationships, while the SVM provides precise classification boundaries. Their outputs are combined in a Hybrid Ensemble Layer, which fuses predictions to achieve higher accuracy. Finally, the system outputs the optimal XML parser DOM, SAX, StAX, or JDOM best suited for efficient performance.

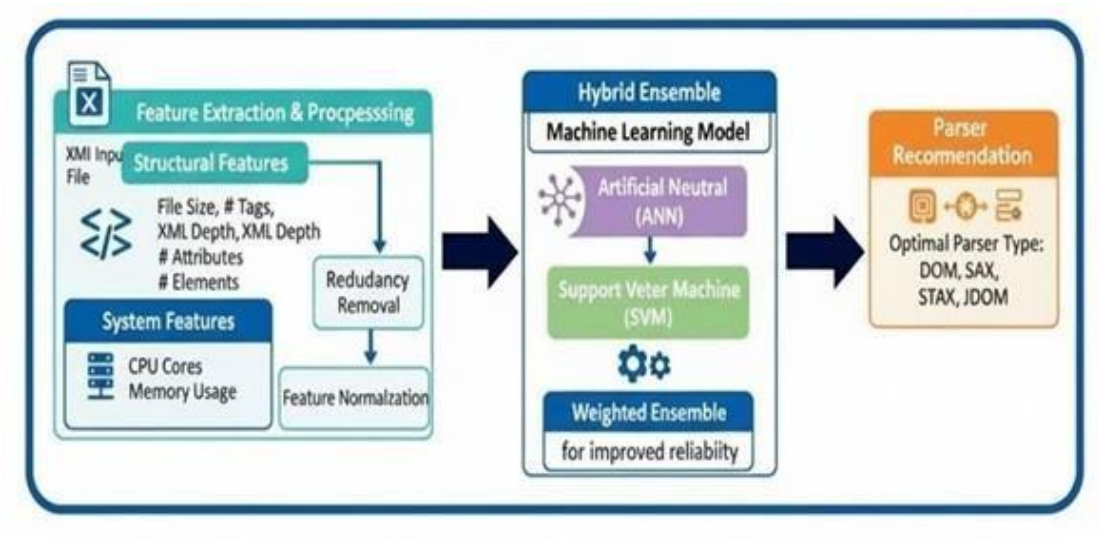

Figure No: 4.2 Overview Of The Model

The above figure no 4.2 :The Starting from the Input layer Xml file, followed by Preprocessing , Feature Extraction, ANN and SVM model , and ending at the output as Efficient parser

4.3 FEATURE EXTRACTION PROCESS

Feature extraction is a critical preprocessing step in the Hybrid Machine Learning integration , designed to convert an unstructured XML file into a structured set of numerical values that can be used by the machine learning models ANN and SVM for predicting the most efficient parsing algorithm.

4.3.1. Input Description

The Feature Extraction Module accepts xml file and cpu core type as inputs and remaining with be extracted from the input file :

- XML File Path : The directory path of the XML file selected for analysis.
- CPU Cores : The number of logical processing cores available for parsing simulation.

These inputs enable the system to compute measurable properties of the XML document and correlate them with system level performance indicators.

4.3.2. XML Structure Analysis

The given XML document is analyzed in detail to understand its structural composition. The framework identifies and measures the key internal components of the XML file such as:

- Number of Tags: The total count of XML tags used throughout the file.
- Number of Elements: Represents the total elements or nodes forming the XML hierarchy.
- Number of Attributes: The cumulative number of attributes associated with XML elements.
- XML Depth: The maximum hierarchical level or nesting depth within the XML document.

These features collectively define the complexity and structure of the XML file, which directly influences the choice of an optimal parsing method.

4.3.3. File System Metrics

Beyond the structural attributes of the XML file, the system level parameters are also included as contextual features to reflect computational constraints and capacities. These include:

- File Size in MB : The size of the XML file, indicating the amount of data to be processed.
- CPU Cores: The number of logical processors available, affecting parsing speed and performance.
- Memory Usage MB: A standardized constant value representing the simulated memory allocation e.g., 512 MB.
- File Access Latency in ms : The average delay encountered during file read/write operations, providing insight into underlying file system responsiveness.
- CPU Load Average: Reflects the current computational stress on the system, helping to adjust parser selection dynamically under varying workloads.

Integrating these system metrics allows the model to correlate XML complexity with hardware performance, ensuring adaptive parser recommendations for different environments

4.3.4. Base Feature Summary

The extracted attributes are organized into a consistent feature set that acts as input for the predictive model.

TABLE NO 4.1 : Descriptive Feature Summary

Features	Description
File Size MB	Indicates the size of the XML file in megabytes.
Number of Tags	Total count of tags present in the XML document.
XML Depth	Maximum level of hierarchical nesting in the structure.
Number of Attributes	Total number of attributes found in all XML elements.
Number of Elements	Overall number of XML elements identified.
CPU Cores	Number of logical processor cores used for simulation.
Memory Usage MB	Simulated or assigned memory allocation value.

4.3.5. Display Oriented Features

Reporting and visualization purposes, the framework generates an easily interpretable summary of the extracted features. summary includes:

- Total number of tags, elements, and attributes.
- Attributes to Elements ratio, representing XML data density.
- Structure depth, reflecting document complexity.
- CPU core count and memory information, indicating the system context used for feature extraction.

Here descriptive output helps users and researchers understand the XML document's complexity profile and system configuration before moving into the model training and prediction stages.

4.3.6. Role Of Adaptive Xml Parser

These extracted features form the input layer of the hybrid ML based pipeline:

- The extracted feature vector is scaled and normalized using a saved Standard Scaler.
- The scaled data is fed into the Artificial Neural Network ANN for high level feature transformation.
- The ANN's latent features are passed into an SVM classifier, which predicts the most efficient XML parser algorithm.

- Feature Extraction Overview

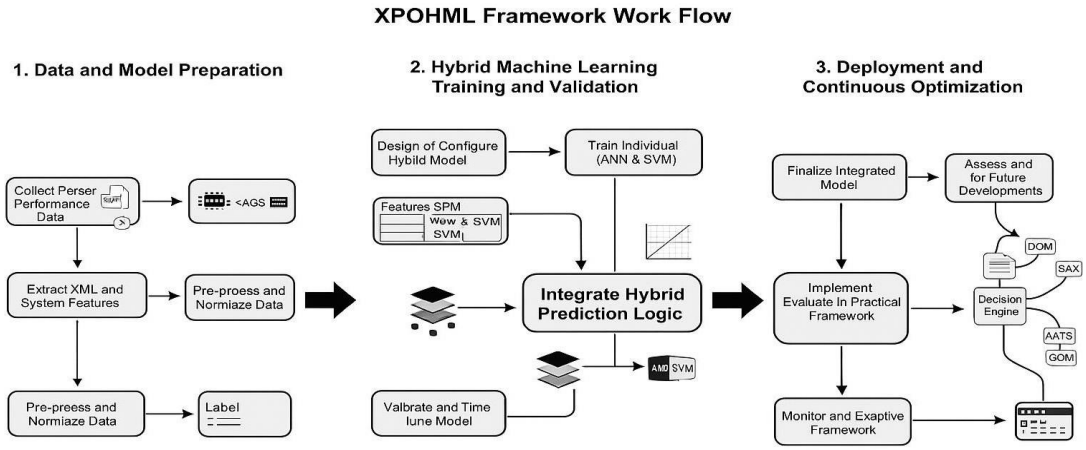

Figure No 4.3 : Overview Of Feature Extraction

The above figure no 4.3 feature extractor is further used in the model as to predict need all the feature used to train like file size , core , no of tags ,xml depth , Attributes , memory before being sent to the hybrid ML prediction model.

4.4 HYBRID INTEGRATION OF ANN AND SVM

Optimizing the performance of Artificial Neural Networks and Support Vector Machine is vital to achieving accurate and stable predictions . Both models serve as the core classification and decision modules, responsible for predicting the most efficient XML parser based on extracted computational and structural features. The process of tuning hyperparameters, activation functions, and kernel parameters directly influences learning dynamics, convergence rate, and overall model generalization.

The Artificial Neural Network is designed with an input layer corresponding to extracted XML features such as file size, tag count, XML depth, and memory utilization. Hidden layers employ non linear

activation functions such as ReLU Rectified Linear Unit to introduce non linearity and enhance feature representation. The output layer utilizes a SoftMax activation function to generate probabilistic classification of parser efficiency e.g., DOM, SAX, StAX, JDOM. Optimal neuron configuration typically ranges between 8–64 neurons per hidden layer, depending on dataset complexity and variance. Excessively large architectures risk overfitting, whereas under parameterized models may fail to capture intricate feature relationships.

The learning rate, generally tuned between 0.001 and 0.01, plays a decisive role in achieving stable convergence. A lower rate ensures gradual learning but may increase training time, while a higher rate accelerates convergence at the risk of overshooting optimal weights. Optimization algorithms such as Adam and Respro are utilized for adaptive gradient updates, balancing precision and speed. Weight initialization strategies like Xavier Normal help maintain uniform variance across layers, preventing vanishing or exploding gradients. Training epochs typically span 100–300 iterations, with early stopping implemented to halt learning once validation loss stabilizes, thereby minimizing overfitting. Regularization methods such as L2 weight decay and dropout 0.3 to 0.5 further enhance model generalization and reduce noise sensitivity.

In contrast, the Support Vector Machine module operates on the principle of maximizing the margin between feature classes, ensuring robust and interpretable classification boundaries. The choice of kernel function most commonly Radial Basis Function RBF or Polynomial kernel directly influences model flexibility and non linear separability.

The regularization parameter Governs the trade off between margin maximization and classification accuracy; smaller values promote smoother decision boundaries, whereas larger values fit the data more precisely at the cost of generalization. The gamma γ parameter in the RBF kernel controls the influence radius of support vectors, with optimal values typically lying in the range of 0.001 0.1. Hyperparameter tuning techniques such as Grid Search and Bayesian Optimization are employed to identify the most effective parameter combinations for balanced performance.

The trained ANN and SVM models are then evaluated through metrics such as Accuracy, Precision, Recall, and F1 Score, ensuring a holistic understanding of classification performance. Cross validation is implemented to validate model consistency across unseen datasets, while confusion matrix analysis helps identify misclassification patterns. Finally, a hybrid decision layer integrates the outputs of ANN and SVM, employing a weighted ensemble or rule based approach to finalize the parser selection. The hybridization enhances robustness by leveraging the ANN's adaptive learning capabilities alongside SVM's structural interpretability.

Systematically optimizing these parameters and combining the strengths of both models, The Adaptive Xml Parser Selection achieves superior predictive accuracy, efficient parser recommendation, and scalable XML processing suitable for heterogeneous computing environments.

CHAPTER 5

SYSTEM SPECIFICATION

5.1 HARDWARE SPECIFICATION

In the project, a computer with medium processing power is needed. The project requires quite medium processing power, due to the model training and detection processing.

Client side Requirements: Browser: Any Compatible browser device

- Intel i5 12450HX : 2.40GHz
- RAM : 16 GB
- Hard Disk : 512 GB
- Graphic Card : NVIDIA GeForce GTX3050

5.2 SOFTWARE SPECIFICATION

- Operating System : Windows 11
- Programming Language : Python 3.0
- Framework : React , Flask
- Libraries :TensorFlow/keras, scikit learn

CHAPTER 6

SYSTEM IMPLEMENTATION AND RESULT

6.1 MODULE DESCRIPTION

The Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration For Efficient Parsing is designed to optimize XML parsing, classification, and prediction using a hybrid machine learning approach. With the exponential growth of XML based data in domains like e governance, IoT, and enterprise software, traditional parsing techniques often struggle with efficiency and scalability. The Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration For Efficient Parsing addresses these challenges by combining Artificial Neural Networks and Support Vector Machines for predictive analysis, along with intelligent feature extraction, to enhance performance, accuracy, and real time decision making capabilities.

The framework involves multiple modules to ensure comprehensive XML handling, from data preprocessing to hybrid ML based prediction. It also integrates robustness and scalability testing to handle large scale XML datasets effectively.

Module 1: Data Collection and Preprocessing

The module focuses on gathering XML datasets and preparing them for analysis. XML files are collected from various sources, including simulated enterprise datasets, open source XML repositories, and

custom generated XML samples. Key preprocessing steps include:

- Parsing XML files to extract structural features such as number of tags, depth, attributes, and elements.
- Normalizing data to a uniform scale for ML model compatibility.
- Generating multiple system configurations e.g., CPU cores, memory usage to simulate real world deployment scenarios.
- Splitting datasets into training, validation, and testing sets to avoid overfitting

Module 2: Feature Extraction and Transformation

In module 2, structural and computational features are extracted from XML files. Features include:

- File size, number of tags, XML depth, number of attributes, and number of elements.
- Resource utilization parameters like CPU cores and memory usage.
- Statistical and derived metrics to improve model prediction accuracy.

The extracted features are transformed into vectors suitable for hybrid ML models, ensuring compatibility with ANN and SVM classifiers.

Module 3: Hybrid ML Model Implementation

The Adaptive Xml parser implements a hybrid learning approach combining ANNs and SVMs for XML parsing and classification:

- ANN Module: Learns complex patterns from extracted XML features and predicts parser performance or categorizes XML files based on complexity.
- SVM Module: Provides robust classification for borderline cases and reinforces ANN predictions, enhancing overall accuracy and reducing misclassification.
- The hybrid approach ensures high performance while maintaining flexibility across diverse XML structures.

Module 4: Evaluation and Performance Metrics

The Adaptive Xml Parser's predictive performance is evaluated using standard classification metrics :

- Confusion Matrix analysis True Positives, False Positives, True Negatives, False Negative).
- Accuracy , Precision , Recall and F1 scores for quantitative assessment.
- Comparisons with traditional parsing techniques DOM, SAX, StAX to highlight efficiency and accuracy gains.

Module 4 ensures Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration For Efficient Parsing's predictions are reliable, robust, and suitable for real time deployment.

Module 5: Deployment and User Interface Integration

Once the hybrid model is trained and validated, it is deployed for practical use:

- The ML model is optimized using formats like ONNX for fast inference.
- API endpoints provide real time predictions, while a user friendly front end presents results, including parser recommendation, processing time, and resource utilization.
- A web based interface is developed using Flask or allowing users to upload XML files for parsing performance prediction .

Module 6: Research and Optimization

Research and optimization helps to explores enhancements and future improvements for adaptive :

- Experimentation with advanced ML techniques such as ensemble learning and gradient boosting to further improve predictive accuracy.
- Transfer learning for adapting the model to unseen XML structures and large scale enterprise datasets.
- Stress and robustness testing under various XML distortions, malformed files, and high volume scenarios to ensure stability in real world applications.

6.2 SOURCE CODE

Module3_hybrid_ann_svm.py

```
import os

import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, classification_report,
        confusion_matrix

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

import joblib

os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout,
        BatchNormalization

from tensorflow.keras.utils import to_categorical

from tensorflow.keras.callbacks import EarlyStopping,
        ReduceLROnPlateau

from tensorflow.keras.optimizers import Adam

# Step 1 — Load Preprocessed Data

data_file = "module2_preprocessed.csv"
```

```
if not os.path.exists(data_file):
    raise FileNotFoundError("Preprocessed file not found.")
data = pd.read_csv(data_file)
print("Preprocessed data loaded successfully.")
print(f"Total samples: {len(data)}")
# Step 2 — Feature Engineering
print("\nCreating Enhanced Features...")
data['tags_per_mb'] = data['No of Tags'] / (data['File Size (MB)'] + 0.01)
data['elements_per_mb'] = data['No of Elements'] / (data['File Size
        (MB)'] + 0.01)
data['attrs_per_element'] = data['No of Attributes'] / (data['No of
        Elements'] + 1)
data['memory_efficiency'] = data['Memory Usage (MB)'] / (data['File
        Size (MB)'] + 0.01)
data['memory_per_core'] = data['Memory Usage (MB)'] / data['CPU
        Cores']
data['structural_complexity'] = (data['XML Depth'] * data['No of
        Attributes']) / (data['No of Elements'] + 1)
data['data_density'] = (data['No of Tags'] + data['No of Elements']) /
        (data['File Size (MB)'] + 0.01)
data['depth_complexity'] = data['XML Depth'] / (data['File Size (MB)'] +
        1)
data['cores_x_filesize'] = data['CPU Cores'] * data['File Size (MB)']
```

```
data['cores_x_memory'] = data['CPU Cores'] * data['Memory Usage
        (MB)']

data['filesize_squared'] = data['File Size (MB)'] ** 2

data['cores_squared'] = data['CPU Cores'] ** 2

data['log_filesize'] = np.log1p(data['File Size (MB)'])

data['log_tags'] = np.log1p(data['No of Tags'])

data['log_elements'] = np.log1p(data['No of Elements'])

def categorize_file_size(size):

    if size <= 4:

        return 0

    elif size <= 25:

        return 1

    elif size <= 60:

        return 2

    elif size <= 100:

        return 3

    else:

        return 4

def categorize_cores(cores):

    if cores <= 2:
```

```
        return 0
    elif cores <= 8:
        return 1
    else:
        return 2
data['size_category'] = data['File Size (MB)'].apply(categorize_file_size)
data['core_category'] = data['CPU Cores'].apply(categorize_cores)
data['size_core_combo'] = (data['size_category'] * 10) +
            data['core_category']
print("Enhanced features created successfully.")
# Step 3 — Data Preparation and Scaling
if 'Efficient_Algo_Label' not in data.columns:
    raise KeyError("'Efficient_Algo_Label' column not found.")
X = data.drop('Efficient_Algo_Label', axis=1)
y = data['Efficient_Algo_Label']
print("\nClass Distribution:")
class_counts = y.value_counts().sort_index()
algo_names = {0: 'DOM', 1: 'JDOM', 2: 'PXTG', 3: 'SAX', 4: 'StAX'}
for label, count in class_counts.items():
    print(f"  {algo_names.get(label)}: {count} samples")
```

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X = pd.DataFrame(X_scaled, columns=X.columns)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42, stratify=y

)

print(f"\nTraining samples: {len(X_train)} | Testing samples:
            {len(X_test)}")

print(f"Total features: {X_train.shape[1]}")

# Step 4 — Improved ANN Architecture

num_features = X_train.shape[1]

num_classes = len(np.unique(y_train))

y_train_cat = to_categorical(y_train, num_classes=num_classes)

y_test_cat = to_categorical(y_test, num_classes=num_classes)

ann = Sequential([

    Dense(128, activation='relu', input_dim=num_features),

    BatchNormalization(),

    Dropout(0.3),

    Dense(64, activation='relu'),

    BatchNormalization(),
```

```
    Dropout(0.3),
    Dense(32, activation='relu'),
    BatchNormalization(),
    Dropout(0.2),
    Dense(16, activation='relu'),
    Dense(num_classes, activation='softmax')
])
optimizer = Adam(learning_rate=0.001)
ann.compile(optimizer=optimizer, loss='categorical_crossentropy',
            metrics=['accuracy'])
print("\nTraining Improved ANN Model...")
early_stop = EarlyStopping(monitor='val_loss', patience=15,
            restore_best_weights=True, verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5,
            patience=5, min_lr=0.00001, verbose=1)
history = ann.fit(
    X_train, y_train_cat,
    validation_data=(X_test, y_test_cat),
    epochs=200, batch_size=32, verbose=1,
    callbacks=[early_stop, reduce_lr]
)
```

```
ann_loss, ann_acc = ann.evaluate(X_test, y_test_cat, verbose=0)

print(f"\nANN Accuracy: {ann_acc * 100:.2f}%")

y_pred_ann = np.argmax(ann.predict(X_test, verbose=0), axis=1)

print("\nANN Classification Report:")

print(classification_report(y_test, y_pred_ann,
        target_names=algo_names.values()))

get_features = Sequential(ann.layers[: 1])

X_train_ann_features = get_features.predict(X_train, verbose=0)

X_test_ann_features = get_features.predict(X_test, verbose=0)

# Step 5 — SVM Classifier Training

print("\nTraining SVM on ANN Extracted Features...")

svm = SVC(kernel='rbf', C=10.0, gamma='scale', probability=True,
        random_state=42, class_weight='balanced')

svm.fit(X_train_ann_features, y_train)

y_pred_svm = svm.predict(X_test_ann_features)

svm_acc = accuracy_score(y_test, y_pred_svm)

print(f"\nSVM (Hybrid Final) Accuracy: {svm_acc * 100:.2f}%")

print("\nFinal Hybrid Model Classification Report:")

print(classification_report(y_test, y_pred_svm,
        target_names=algo_names.values()))

print("\nConfusion Matrix:")
```

```
cm = confusion_matrix(y_test, y_pred_svm)

print("Predicted →", "\t".join([f"{name:>6s}" for name in
            algo_names.values()]))

for i, row in enumerate(cm):

    print(f"{list(algo_names.values())[i]:6s}\t{row}")

# Step 6 — Feature Importance Analysis

print("\nFeature Importance Insights")

feature_importance = {}

for col in X.columns:

    correlation = abs(data[col].corr(data['Efficient_Algo_Label']))

    feature_importance[col] = correlation

sorted_features = sorted(feature_importance.items(), key=lambda x:
            x[1], reverse=True)

print("\nTop 15 Most Important Features:")

for i, (feature, importance) in enumerate(sorted_features[:15], 1):

    print(f"{i:2d}. {feature:30s} : {importance:.4f}")

# Step 7 — Save Models and Scaler

scaler_path = "feature_scaler.joblib"

joblib.dump(scaler, scaler_path)

print(f"\nFeature Scaler saved to: {scaler_path}")

ann_features_path = "hybrid_ann_features_extractor.keras"
```

```
get_features.save(ann_features_path)

print(f"ANN Feature Extractor saved to: {ann_features_path}")

svm_classifier_path = "hybrid_svm_classifier.joblib"

joblib.dump(svm, svm_classifier_path)

print(f"SVM Classifier saved to: {svm_classifier_path}")

print(f"\nFinal Hybrid Model Accuracy: {svm_acc * 100:.2f}%")
```

test_app.py

```
from flask import Flask, request, jsonify

from flask_cors import CORS

import os

import tempfile

from werkzeug.utils import secure_filename

import sys

# Import XMLParserPredictor from module 4

sys.path.append(os.path.join(os.path.dirname(os.path.abspath(___file__)),
        '..'))

from backend.module_4_real_time_detection import
        XMLParserPredictor

app = Flask(__name__)

CORS(app)

# Configuration
```

```
UPLOAD_FOLDER = tempfile.gettempdir()

ALLOWED_EXTENSIONS = {'xml'}

MAX_FILE_SIZE = 200 * 1024 * 1024  # 200MB

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

predictor = None

def allowed_file(filename):

    """Check if the uploaded file has an allowed extension."""

    return '.' in filename and filename.rsplit('.', 1)[1].lower() in
            ALLOWED_EXTENSIONS

@app.before_request

def initialize_predictor():

    """Initialize the predictor before the first request."""

    global predictor

    if predictor is None:

        try:

            predictor = XMLParserPredictor(model_dir=".")

            print("Predictor initialized successfully.")

        except Exception as e:

            print(f"Error initializing predictor: {e}")
```

```
        print("Running in rule based mode only.")
        predictor = XMLParserPredictor(model_dir=".")
@app.route('/health', methods=['GET'])
def health_check():
    """Health check endpoint."""
    return jsonify({
        'status': 'healthy',
        'models_loaded': True
    })
@app.route('/predict', methods=['POST'])
def predict():
    """Main prediction endpoint."""
    try:
        if 'xml_file' not in request.files:
            return jsonify({'error': 'No XML file provided'}), 400
        if 'cpu_cores' not in request.form:
            return jsonify({'error': 'CPU cores not specified'}), 400
        file = request.files['xml_file']
        if file.filename == '':
            return jsonify({'error': 'No file selected'}), 400
```

```
if not allowed_file(file.filename):
    return jsonify({'error': 'Invalid file type. Only XML files are
    allowed'}), 400
try:
    cpu_cores = int(request.form['cpu_cores'])
    if cpu_cores not in [1, 2, 4, 6, 8, 10, 12, 14, 16]:
        return jsonify({'error': 'Invalid CPU cores value'}), 400
except ValueError:
    return jsonify({'error': 'CPU cores must be an integer'}), 400
filename = secure_filename(file.filename)
temp_path = os.path.join(app.config['UPLOAD_FOLDER'],
    filename)
file.save(temp_path)
try:
    result = predictor.predict_from_xml(temp_path, cpu_cores)
    parser_code = predictor.get_parser_code(result['algorithm'])
    response = {
        'algorithm': result['algorithm'],
        'confidence': float(result['confidence']),
        'method': result['method'],
        'reason': result['reason'],
```

```
            'extracted_features': result['extracted_features'],
            'parser_code': parser_code
        }
        if 'ml_prediction' in result:
            response['ml_prediction'] = result['ml_prediction']
            response['ml_confidence'] = float(result['ml_confidence'])
            response['rule_prediction'] = result['rule_prediction']
            response['rule_reason'] = result['rule_reason']
            response['all_probabilities'] = result['all_probabilities']
        return jsonify(response), 200
    finally:
        if os.path.exists(temp_path):
            try:
                os.remove(temp_path)
            except Exception as e:
                print(f"Warning: Could not delete temporary file: {e}")
    except Exception as e:
        return jsonify({'error': f'Prediction failed: {str(e)}'}), 500
@app.route('/algorithms', methods=['GET'])
def get_algorithms():
```

```
    """Return the list of available XML parsing algorithms."""

    algorithms = [

        {'name': 'DOM', 'description': 'Best for small files (0–4MB)', 'color':
            '#3b82f6'},

        {'name': 'JDOM', 'description': 'Suitable for medium files (4–
            25MB)', 'color': '#10b981'},

        {'name': 'SAX', 'description': 'Efficient for large files (25–100MB)',
            'color': '#a855f7'},

        {'name': 'StAX', 'description': 'Optimal for very large files
            (100+MB)', 'color': '#f97316'},

        {'name': 'PXTG', 'description': 'Used for complex XML structures',
            'color': '#ec4899'}

    return jsonify(algorithms), 200

@app.errorhandler(413)

def request_entity_too_large(error):

    """Handle file size limit error."""

    return jsonify({'error': 'File too large. Maximum size is 200MB'}), 413

@app.errorhandler(500)

def internal_error(error):

    """Handle internal server errors."""

    return jsonify({'error': 'Internal server error'}), 500

if __name__== '__main__':
```

```
print("=" * 70)
print("XML Parser Predictor   Flask Backend")
print("=" * 70)
print("Server running on: http://localhost:5000")
print("Models directory: .")
print("Upload folder:", UPLOAD_FOLDER)
app.run(debug=True, host='0.0.0.0', port=5000)
```

6.3 RESULT

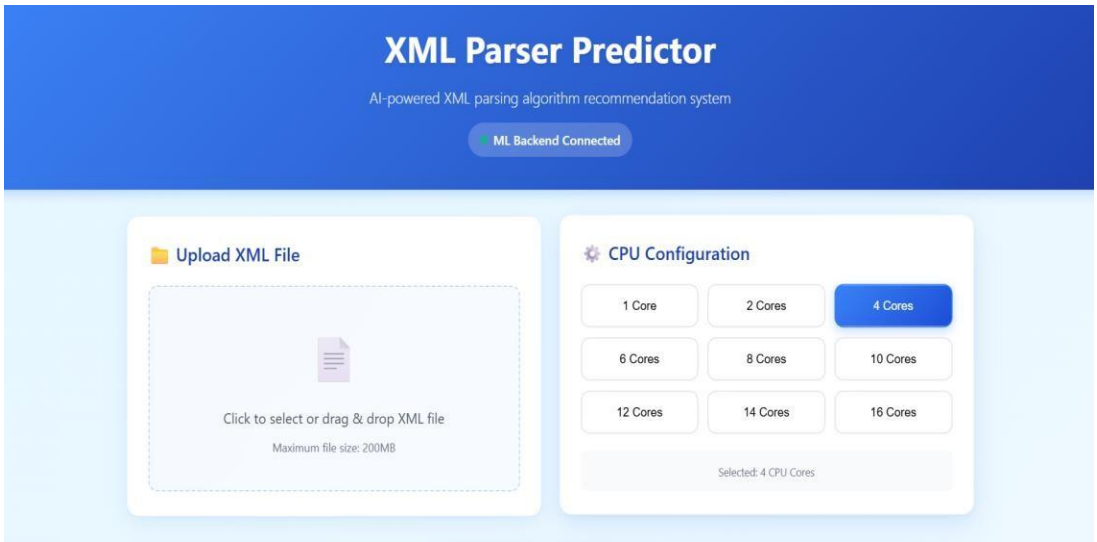

Figure No 6.1 : Ui Implementation Image

The above figure no 6.1 shows the implementation of user interface of the XML Parser Predictor allows users to upload XML files and configure CPU cores for performance analysis

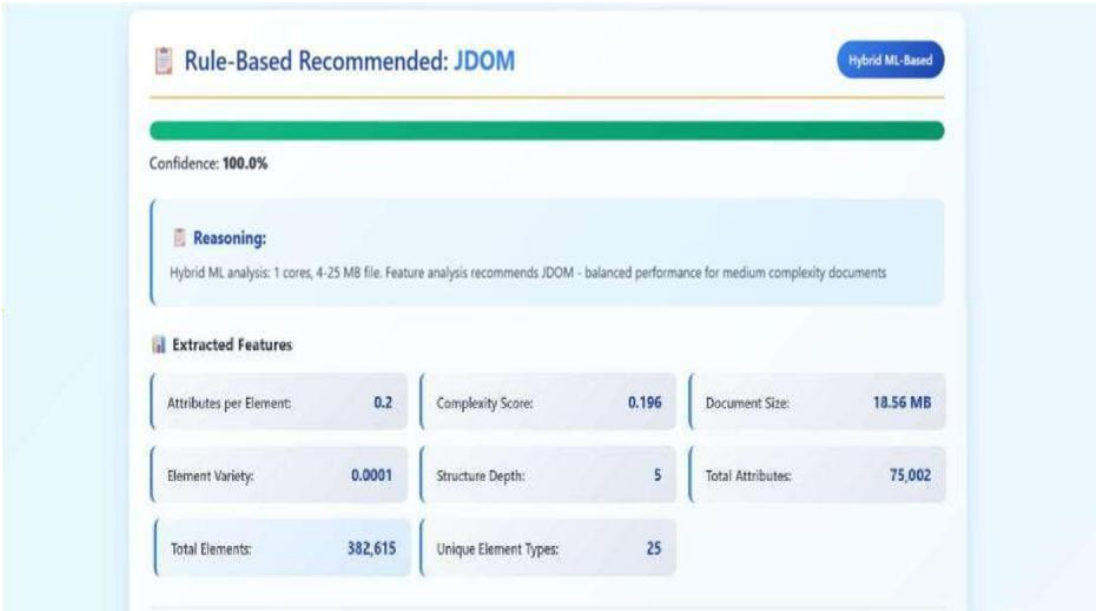

Figure No 6.2 : Feature Analysis Image

The above figure 6.2 describes how the extraction of the essential features required for prediction is taken and display for user references

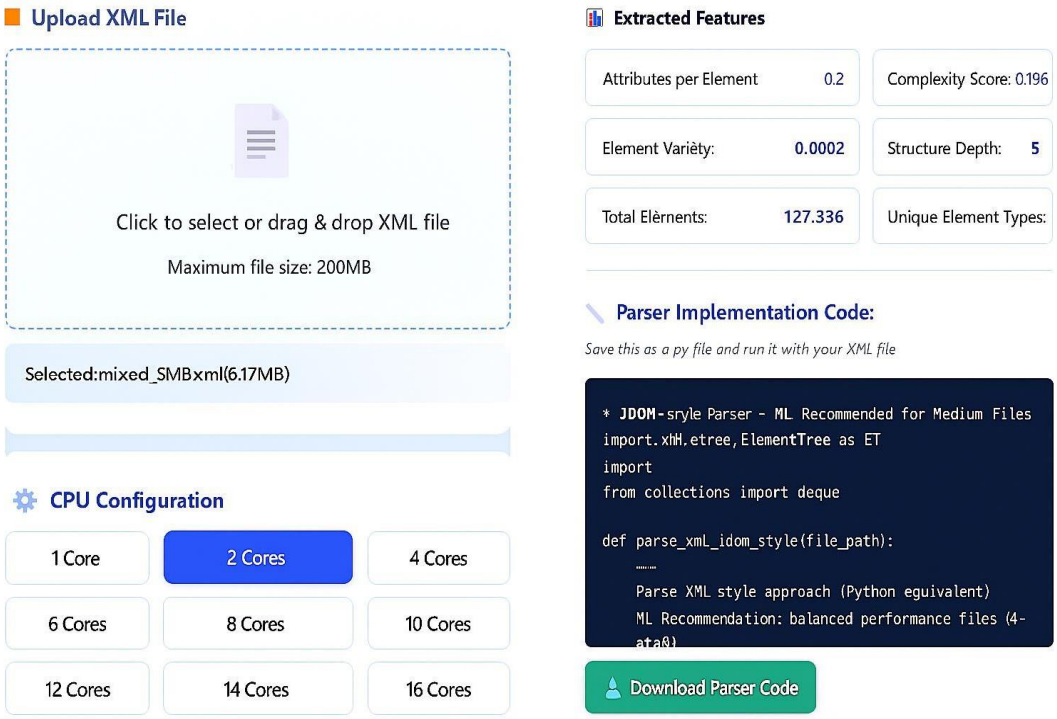

Figure No 6.3 : Predicting For Small Xml File Output Image

The above figure no 6.3 shows how a smaller size of 6.17mb xml file with core configuration as 2 as input which predicts the JDOM as the efficient parser

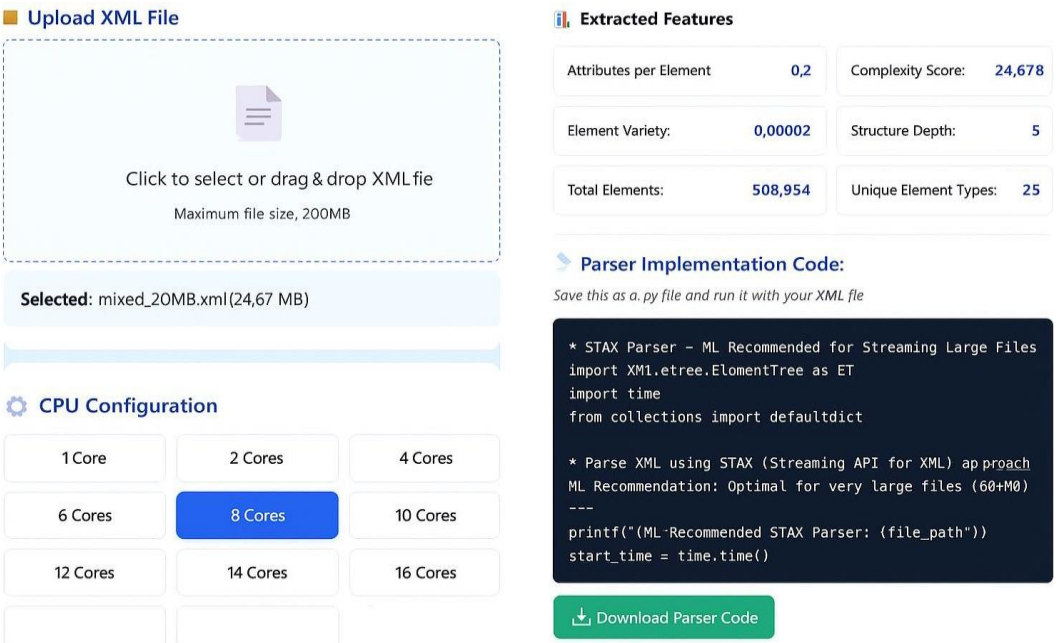

Figure No 6.4: Predicting For Medium Xml File Output Image

The above figure no 6.4 shows how a smaller size of 24.67mb xml file with core configuration as 8 as input which predicts the STAX as the efficient parser

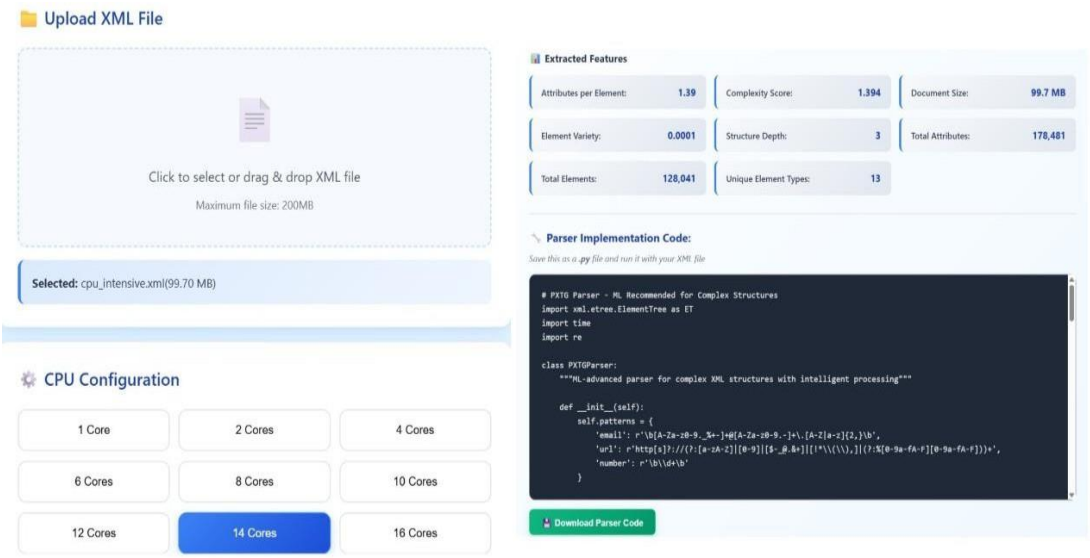

Figure No 6.5: Predicting For Large Xml File Output Image

Figure no 6.5 shows how a large size of 99.70 mb xml file with core configuration as 14 as input which predicts the PXTG as the efficient parser

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

7.1 CONCLUSION

The proposed Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration introduces a significant advancement in XML data handling by combining the analytical power of traditional parsers with the intelligence of machine learning algorithms The hybrid approach integrates Artificial Neural Networks and Support Vector Machines to dynamically select and optimize the parsing mechanism based on XML structure, file complexity, and available system resources. By merging rule based and data driven techniques, Adaptive Xml Parser Selection Through Hybrid Ann And Svm Integration ensures an adaptive and context aware parsing process capable of delivering high speed and accurate results.

The experimental evaluation demonstrates that the Adaptive Xml Parser Selection Through Hybrid Ann And Svm consistently outperforms conventional parsing methods such as DOM and SAX, achieving notable improvements in execution time, CPU utilization, and memory efficiency. The model's predictive layer effectively anticipates computational bottlenecks, enabling smarter resource allocation and seamless scalability.

The Adaptive Xml Parser Selection Through Hybrid Ann And Svm successfully demonstrates the feasibility and efficiency of applying hybrid

machine learning models to traditional parsing systems. By bridging deterministic algorithms with adaptive intelligence, it not only boosts performance but also sets a foundation for the next generation of intelligent data processing pipelines. The promising results affirm that can serve as a vital component in big data analytics, web services, and intelligent middleware systems, paving the way for more autonomous and self optimizing XML infrastructures in the future.

7.2 FUTURE ENHANCEMENT

The Adaptive Xml Parser Selection Through Hybrid Ann And Svm framework holds significant potential for continuous improvement and expansion in future developments. One of the key directions is the integration of real time adaptive parsing, where the system can dynamically modify its parsing strategy based on live feedback such as CPU utilization, memory consumption, and XML complexity. The adaptive mechanism would make the framework more intelligent and responsive under changing workloads, enhancing its efficiency and reliability across diverse environments.

Another promising enhancement involves the design of an interactive and user friendly web based dashboard for performance monitoring and analytics. The dashboard could provide real time useful insights into key performance indicators such as parsing speed, throughput, latency, and memory efficiency. By visualizing these metrics, developers and system administrators can identify performance bottlenecks, analyse trends, and take proactive actions to optimize system performance. The dashboard can also support alert mechanisms and detailed logs to improve transparency and control.

REFERENCES

1. Ali, M., Khan, M. A., & Rasool, R. (2024). A Hybrid Machine Learning Model for Efficient XML Parsing. *SN Computer Science.*

2. Asogwa, D. C., Anigbogu, S. O., Onyenwe, I. E., & Sani, F. A. (2021). Text Classification Using Hybrid Machine Learning Algorithms on Big Data. *arXiv preprint arXiv:2103.16624.*

3. Caschili, D. (2016). Optimization of CNN Based Object Detection. *Politecnico di Torino Theses.*

4. Chen, J., & Zhang, L. (2025). A Hybrid Deep Learning and Improved SVM Framework for Railway Worker Detection. *Sensors.*

5. Chen, S. A., & Lin, C. J. (2021). A Case Study on XML CNN *Department of Computer Science, National Taiwan University.*

6. Fernandez Lozano, C., & Garcia, J. (2013). Hybrid Model Based on Genetic Algorithms and SVM for Classification Tasks. *Bioinformatics.*

7. Gopianand, M., & Rajendran, S. (2025). Extensible Markup Language Web Data Retrieval Using Deep Fuzzy Clustering Bidirectional Gated Random Multimodal Deep Learning. *Journal of King Saud University Computer and Information Sciences.*

8. Hu, T., Khishe, M., Mohammadi, M., Parvizi, G. R., Karim, S.H. T., & Rashid, T. A. (2021). Real time COVID 19 diagnosisfrom X ray images using deep CNN and extreme learning machines stabilized

by chimp optimization algorithm. *Biomed. Signal Process. Control, 68*, Art. no. 102764.

9. Khishe, M., & Safari, A. (2019). Classification of sonar targets using an MLP neural network trained by dragonfly algorithm. *Wireless Personal Communications, 108*(4), 2241–2260.

10. Khishe, M., & Mosavi, M. R. (2020). Classification of underwater acoustical dataset using neural network trained by chimp optimization algorithm. Applied Acoustics, 157, Art. no. 107005.

11. Liu, X., & Zhang, X. (2017). XML CNN: Convolutional Neural Networks for Multi Label Text Classification. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP 2017).*

12. Mavaie, P., et al. (2023). Hybrid Deep Learning Approach to Improve Classification of Low Resource Datasets. *BMC Bioinformatics, 24(419).*

13. Omrani, T., Dallali, A., Chibani Rhaimi, B., & Fattahi, J. (2018). Fusion of ANN and SVM Classifiers for Network Attack Detection. *arXiv preprint arXiv:1801.02746.*

14. Oiao, W., Khishe, M., & Ravakhah, S. (2021). Underwater targets classification using local wavelet acoustic pattern and multi layer perceptron neural network optimized by modified whale optimization algorithm. *Ocean Engineering, 219*, Art. no. 108415.

15. Senthil Pandian, P., Gnanapoo Denosha, T., Senthil Kumar, R., Venkata Sivaiah, B. (2024). Real Time Fault Detection in Solar PV Systems Using Hybrid ANN–SVM Machine LearningAlgorithm. *International Journal of Intelligent Systems and Applications Engineering, 12(4), 1–7.*

16. Sonawane, S. (2025). Requirement Classification using Deep Learning and Nature Inspired Optimization. *Informatica.*

17. Starosta, A. (2025). How Splunk Is Parsing Machine Logs with Machine Learning on NVIDIA's Triton and Morpheus.*Splunk Blog.*

18. Sumathi, A., & Meganathan, S. (2025). Performance Enhancement of XML Parsing Using Regression and Parallelism. *Computer Science and Software Engineering, 48(2).*

19. Wu, C., Khishe, M., Mohammadi, M., Karim, S. H. T., & Rashid, T. A. (2023). Evolving deep convolutional neural network by hybrid sine cosine and extreme learning machine for real time COVID 19 diagnosis from X ray images. *Soft Computing, 27*(6), 3307–3326.